

面试押题 04 | OE 异步状态算子

更新：2026-09-29。对应九坤实习中的“OE 异步状态算子”。本文是个人面试准备材料，性能数字按原始实验口径保留。

0. 开源仓库技术介绍与应用

vLLM 将请求调度、KV cache 管理、模型执行与采样组织成推理服务；连续批处理允许请求不断加入和退出，模型执行中的 batch 位置并不是永久请求身份。

典型应用：多请求在线服务与批量生成。本题是在内部模型接入 vLLM 时优化 OE 历史状态：将依赖移到 GPU，同时处理请求重排、恢复与 slot 复用。OE 是模型侧机制，不能说成 vLLM 上游普遍具有的标准层。

1. 这个模块在做什么

OE (Over-Encoding) 是模型已有的历史 token 编码模块：将当前 token 和最近若干 token 组成 n-gram，计算多项式 hash 得到 `oe_input_ids`，再经过 embedding、projection 和混合形成模型输入的一部分。我没有发明 OE 模型结构；负责的是 vLLM 中的异步执行、GPU hash、跨 step 请求状态和多卡正确性。

普通 async scheduling 可以在 CPU 不等待本轮采样 token 回传时准备下一步，但 OE 依赖真实 token history。若仍在 CPU 计算历史 hash，就会把“GPU 采样 → token 回 CPU → CPU hash → ids 回 GPU”串回关键路径，抵消异步调度收益。

本项目有两个相互独立的问题：一是同步和小 kernel 的性能开销；二是 request 身份、slot、batch 位置变化引起的跨 step 状态错误。不能把二者都归结为“写一个融合 kernel”。

2. STAR 口述版（约 90 秒）

S | 背景。模型的 OE 输入依赖最近 token，但 vLLM 的异步调度希望避免每步等待 token 回 CPU。旧路径在历史构造和回传处重新引入同步，多卡 sync/async 对拍还会出现 token 分叉。

T | 任务。我负责把 decode 的 OE 历史和 hash 准备放到 GPU，且不破坏请求切换、恢复、slot 复用、CUDA Graph 和 Tensor Parallel 的正确性。

A | 行动。我实现 Triton fused-hash，用 GPU 上真实 sampled token 和最近历史生成下一步 OE ids；按 request 保存恢复历史，处理首次 decode 的 prompt suffix 初始化，并修正 slot reorder 后的索引。随后用 CPU integer golden、逐 token 对拍和 TP1/2/4 矩阵分层验证，发现 TP>1 的一部分分叉来自 batch-sensitive 浮点路径，因此让 sync/async 都走 BI，并保留 async-unfused 的多卡安全路径。

R | 结果。TP1 fused+BI 的 28 个 decode case 全通过，平均吞吐相对同步路径提升 4.99%；TP2/TP4 各三轮 28 个 case 均零 mismatch，平均 decode 吞吐分别提升约 4.6%/3.0%。收益主要在 decode；prefill 接近持平，mixed 短插入窗口曾退化，因此我没有把所有阶段都写成加速。

3. 数据流与状态为什么难

```
step N: GPU forward → GPU sampling → sampled_token_ids
      | 不等待 token 回 CPU
step N+1 CPU: request 调度 → prev/current slot 映射 → metadata
      |
step N+1 GPU: scatter 当前 token → 读取旧历史 → 计算 hash
      → 写 oe_input_ids → 推进 request history → model forward
```

具体实现必须保证 hash 消费的是对应请求的旧上下文和本步真实 token，且历史推进恰好一次；不能先覆盖旧历史，再误把新历史当旧上下文读取。metadata、history 和 OE ids 使用可重复利用的稳定 buffer，以满足编译/Graph 的生命周期要求。

场景	如果只用 batch 下标会怎样	已实现的处理
prefill 刚结束，首个 async decode	历史未初始化或含占位 token	从真实 prompt suffix 刷新 GPU 历史
请求被驱逐后恢复	slot 不在原位置，历史丢失	<code>oe_recent_token_ids_by_req</code> 按 request 保存/恢复
slot 复用	新请求读到旧请求历史	生命周期与 request 身份绑定，不把 slot 当永久身份
batch 重排	prev index 与 current index 不一致	明确 prev/current 映射，读取用 current request index
mixed prefill+decode	CPU 不知道 async decode 的真实新 token	CPU 填 prefill 部分，GPU 覆盖 decode 对应位置

这里的“状态管理”不是 KV cache 的分页管理；OE history 是额外的小型请求状态，但错误会改变模型下一步实际输入，因此不能因为状态小就降低验证要求。

4. 交付方案与没有交付的方案

交付路径：TP1 使用 fused-hash；TP>1 默认 async-unfused GPU OE + BI，并约束到兼容的 attention backend。`OEEmbedding` 拆分为独立 CustomOp，保持 forward 数学和参数层级，不把 Python 场景分支泄漏到编译图。

没有把完整 OE forward fusion 当作本次成功。 曾尝试把 lookup/proj/add/div 也一起融合。部分 precompute 方案正确但比 hash-only 慢；in-model compiled-safe 尝试曾出现输出不一致；full-preembed 为兼容 Graph 仍需产生 OE ids，额外做了重复工作。关闭 Graph 虽能让某些实现通过，却导致模型主路径严重退化，因此不能用关闭 Graph 的结果宣布融合成功。

BI 不是免费的性能开关。 TP>1 的 sync/async 执行轨迹会影响 batch shape、attention/reduction 路径，低位差异可能触发近似并列 token 的选择翻转；OE 再将该 token 写入历史，后续输入真正变了。两侧同时启用 BI 可以在已测矩阵中保持 token 一致，但有约 5% 的性能代价。

5. 验证矩阵与结果口径

基础 decode 矩阵为 `batch=1/2/4/8/12/16/20 × input_len=1024/2048/4096/8192`，每组 28 case。主正确性是与同步 baseline 逐 token exact match，不是仅用最终文本相似度。

配置	正确性	吞吐与范围
TP1 fused, BI ON	28/28, 零 mismatch	平均 token/s 比 1.04989966, 约 +4.99%
TP1 fused, BI OFF	27/28	约 +10.3%, 但未全过正确性, 不能作为交付主数字
TP2 async-unfused, BI ON	$28/28 \times 3 = 84/84$	final decode 平均 token/s 比 1.046
TP4 async-unfused, BI ON	$28/28 \times 3 = 84/84$	final decode 平均 token/s 比 1.030; 26/28 更快
TP2/TP4 prefill, BI ON	各三轮 28/28	接近持平, 不是主要收益来源
TP2/TP4 mixed, BI ON	各三轮 28/28	特定短插入窗口约慢 6%/10%

“84/84”是 28 个配置重复三次，不是 84 个不同 shape；TP4 的均值为正，不等于每个 case 都更快。TP2 unseen stability 的 12 case $\times$ 3 repeats 也全部通过，支持扩展稳定性，但不替代任意生产流量验收。

后续 Router $\times$ OE 联合测试还做了 150 个 fused hash 对 CPU integer golden 的逐位核验。这里要与“auto Router 的 mixed 路径曾失配”区分：那轮 async+persistent 控制组通过，不能把联合问题直接归咎于 OE hash。

6. 高频问题与直接回答

Q1：为什么 hash 放 GPU 反而有可能变慢？

hash 本身计算量小，如果拆成多次 gather、scatter、乘加、取模和 per-head launch，调度开销会超过计算。融合的价值是减少热路径依赖与 launch 数量，而不是 GPU 的单次整数操作必然比 CPU 快；小 batch、短上下文尤其要看端到端 wall。

Q2：多项式 hash 怎么证明跟 CPU 一致？

首先保持 token 顺序、边界 padding、整数宽度和取模规则一致，用 CPU integer golden 对拍；再检查 GPU 输出 ids，而非只看生成文本。在后续联合验收中 150 个直接 case 逐位通过，这比仅观察最终 token 更能隔离 hash 本身。

Q3：slot 和 request ID 有什么区别？

request ID 是逻辑请求身份，slot 是当前调度布局中的位置，可能被换出、重排和复用。历史归属必须按 request 管理，GPU 执行时再映射到当前 slot；否则 batch 变化会让一个请求继承另一个请求的历史。

Q4：为什么 prefill $\rightarrow$ decode 首步特别容易出错？

prefill 的历史主要来自 CPU 已知 prompt，而后续 decode 历史在 GPU 持续推进；首步是两套来源的交接点。如果用未刷新的 GPU buffer 或占位 token 生成 hash，会在第一步就改变 OE 输入，因此要用 prompt suffix 正确初始化。

Q5：为什么 mixed 时还保留 CPU 路径？

新 prefill 的 token 是 CPU 已知输入，async decode 的新 token 尚在 GPU。交付方案按这两类来源准备数据：CPU 生成/填充 prefill，GPU 只覆盖 decode 位置。这样不为尚未拿到的 token 强迫同步，也不假设整个 batch 都是 decode。

Q6：TP rank 会生成不同的 OE ids 吗？

在 token history、索引和整数公式一致时，hash 应得到同一组全局 OE ids；之后 VocabParallelEmbedding 负责分片与规约。但数学上可行不等于这版 fused 接入已在 TP>1 达到交付条件，所以最终采用 async-unfused+BI，多卡收益不能写成来自 TP1 fused kernel。

Q7：BI 为什么要同时给 baseline 开？

目标是在相同数值契约下比较 sync/async。如果仅给 optimized 开 BI，baseline 仍走 batch-sensitive 路径，两者本来就可能产生不同 token，oracle 不对齐。性能也必须在相同 BI 条件下比较，不能用取消正确性保护的配置抬高收益。

Q8：token 分叉一定表示 hash 错误吗？

不一定。先定位首个不同量：history、hash ids、embedding、logits，再看采样。若 history 与 hash 点值一致，而近并列 logits 因浮点路径变化造成 token 分叉，这是数值路径问题；下一步 OE 再放大差异。不能只看最终文本倒推 hash 实现错误。

Q9: CustomOp 是否就能保证 Graph-safe?

不能。CustomOp 只是提供编译边界，还要保证 mutation 声明、输入输出地址、buffer 生命周期、每步数据更新与真实 kernel 依赖都正确。历史 full-preembed 尝试就是提醒：注册了黑盒 op，不等于 replay 一定消费了本步新 ids。

Q10: debug checker 报错能不能直接判断生产实现错?

不能跳过检查器本身的状态。后续联合验收曾发现 fused pure-decode 更新 GPU history，但 CPU shadow 未同步，切到 mixed 时产生 debug 假阳性；所以既要独立 golden 验证，也要确认 shadow 使用相同逻辑步的真实历史。debug 开销也不能进入性能计时。

Q11: 为什么 TP4 的收益比 TP2 小?

省掉的 CPU 同步/历史准备是局部成本；TP 增大后通信、规约和 BI 代价在总时延中的占比变化，局部优化收益会被稀释。我的证据是已测矩阵均值和逐 case 分布，不把它外推为关于 TP 的普遍单调规律。

Q12: 如果继续优化，你优先做什么?

先补请求换出恢复、mixed、Graph replay 和检查器的状态契约，再评估 TP>1 fused-hash 是否能独立过门槛；性能上优先消除重复 hash/预计算，而不是为了减少 kernel 数强行融合完整 embedding/projection。任何新路径先过 token/history 正确性，再做性能矩阵。

7. 追问边界与易错表述

- 可以说“GPU fused-hash + request 状态治理”，不要说“发明 OE 架构”。
- TP2/TP4 交付是 async-unfused + BI；早期设计文档写“支持 TP>1 fused”不等于最终默认启用。
- 4.99%、4.6%、3.0% 是指定 decode 矩阵的均值，不是所有 online serving 请求的固定提升。
- 完整 forward fusion 有失败/退化实验，应主动说明取舍；不能把实验代码存在说成正式交付。

8. 开发记录与证据索引

- 最终交付口径与实现点： /volume/pt-train/users/zhaoye/doc/oe/pr_description_fused_oe.md。
- 使用与 TP 回退： /volume/pt-train/users/zhaoye/doc/oe/OE_async_使用说明.md。
- 多轮正确性与阶段边界： /volume/pt-train/users/zhaoye/doc/oe/OE_batch_invariant_最新结论_20260707.md（正文更新至 7 月 10 日）。
- 原始设计与数据流： /volume/pt-train/users/zhaoye/doc/oe/OE算子实现与数据流详解_20260703.md；涉及 TP fused 的早期描述以最终交付文档为准。
- 失败尝试和 profile： /volume/pt-train/users/zhaoye/doc/oe/OE_async_开发简报_20260701.md，附录 E/F。
- TP1 +4.99% 原始汇总： /volume/pt-train/users/zhaoye/doc/oe/assets/decode/tp1_fused_unfused/oe_decode_tp1_fused_unfused_overview_20260709.csv。
- 150 个 CPU golden、debug shadow 边界： /volume/pt-train/users/zhaoye/doc/oe/BI_RouterGEMM_OE联合验收_20260728.md。
- 联合 mixed 问题后续定位： /volume/pt-train/users/zhaoye/doc/oe/BI_RouterGEMM_OE全面性能测试执行记录_20260729.md。